

Solution for Better Data Transfer

A prefix-free code over the three characters $\emptyset$, 1 , $*$ is exactly a rooted tree in which every node has up to three children, reached by an edge labelled $\emptyset$ (cost 1), 1 (cost 1), or $*$ (cost h). The codewords are the root-to-leaf label strings, one leaf per symbol, and the cost of a leaf is the total edge cost on its path from the root — its *depth*, measured in nanoseconds. If symbol i occurs a_i times, the message takes $\sum_i a_i \cdot (\text{depth of } i)$ nanoseconds to transmit, so we want a tree with s leaves that minimizes the weighted sum of leaf depths.

Two facts hold regardless of h :

1. **Sorted assignment.** Only the multiset of leaf depths matters. Given the depths, an optimal assignment puts the heaviest symbol on the shallowest leaf. This can be proven by a simple exchange argument: if a heavier symbol sat deeper than a lighter one, swapping them would improve the total transmission cost. Hence, if we knew the shape of the tree, we could just sort the weights in decreasing order and fill them into leaves from top to bottom.
2. **No single-child internal nodes.** In an optimal tree every internal node has at least two children: a node with a single child can be contracted — i.e., we can remove it and instead attach its children directly to its parent. This would strictly decrease the depth of every leaf below it. Hence an optimal tree has at most $s - 1$ internal nodes.

The corner case $s = 1$ is handled separately: the one codeword must be non-empty, and assigning the symbol $\emptyset$ as the codeword is clearly always optimal. This was helpfully covered by an example. Below we'll assume that $s \geq 2$.

Case $h = 1$: ternary Huffman code

When $h = 1$, all three characters cost 1, so we are just looking for an optimal Huffman code with a three-letter alphabet.

A ternary merge tree reduces the number of active items by 2 with each merge and must end at a single root, so the number of leaves has to be odd. If s is even we therefore add one dummy symbol of weight 0 so that we don't have to handle special cases.

It can now be shown that there is always an optimal tree in which each inner node is full, i.e., has all three children. (The proof is again via switching arguments: any other tree can be rearranged into this shape without increasing the total cost.)

The resulting algorithm is then greedy: repeatedly remove the three smallest weights from a min-heap, merge them into one node whose weight is their sum, add that sum to the answer, and push the new element back onto the heap. Stop when just one node remains. Each symbol's weight is added once per level above it, so the total of all merge sums equals the weighted depth sum.

Correctness follows from the standard Huffman argument adapted to degree 3. Once we know that, we know that the three lightest elements will all end up in the bottommost layer of the tree, and without loss of generality we may assume that they are siblings. We can now merge them together and create one new symbol whose weight is the total weight of the three original symbols. All that remains is constructing the

optimal tree for this new set of $n - 2$ symbols, and that can be done inductively – by repeating the same argument.

As this construction can explicitly construct the tree, we can then just read the codewords off the paths from its root to its leaves.

This runs in $O(s \log s)$ per test and covers subproblem **B3**.

Case $h = 2$: dynamic programming

When $*$ costs 2, the three edges no longer have equal cost, and no Huffman-style greedy is known. Instead we are going to explicitly optimize over the *shape* of the tree, described level by level.

For a fixed tree and each level $\ell \geq 0$ (depth in nanoseconds) let n_ℓ be the number of leaves at depth ℓ and m_ℓ the number of internal nodes at depth ℓ . Every node at level ℓ is either a $0/1$ -child of an internal node at level $\ell - 1$ or the $*$ -child of an internal node at level $\ell - 2$, so

$$n_\ell + m_\ell \leq 2m_{\ell-1} + m_{\ell-2},$$

with $m_0 = 1$ (the root is internal, as $s \geq 2$) and $m_{-1} = 0$. Conversely, **any** sequence satisfying these inequalities with $\sum_\ell n_\ell = s$ comes from some tree: process levels top-down, and at each level there are exactly $2m_{\ell-1} + m_{\ell-2}$ child slots available, which by the inequality is enough to host the required $n_\ell + m_\ell$ nodes. Internal nodes that end up childless are pruned without changing any leaf depth. So minimizing over trees is exactly minimizing over such profiles.

The first fact from the introduction allows us to calculate the cost of any such profile: Sort the weights decreasingly, let A_p be the sum of the p largest and $W = A_s$ the total. Charge the cost one level at a time: advancing from level ℓ to $\ell + 1$ costs $W - A_p$, where p is the number of symbols already placed at levels $\leq \ell$, because each not-yet-placed symbol gains one more level of depth. Summing these charges reproduces $\sum_i a_i \cdot (\text{depth})$ exactly.

This suggests a dynamic programming algorithm whose state after deciding level ℓ is

$$(p, u, v) = (\text{symbolsplaced}, m_{\ell-1}, m_\ell).$$

The level index ℓ itself is not part of the state – the transition cost $W - A_p$ does not depend on it. We start in $(0, 0, 1)$ with cost 0. From (p, u, v) the next level has capacity $c = 2v + u$ slots; a transition costs $W - A_p$ and chooses how many $n \in \{0, \dots, c\}$ of them become leaves, moving to $(p + n, v, c - n)$. Two observations keep this small and correct:

- **Finish as soon as possible.** If $c \geq s - p$ we can place all remaining symbols as leaves on the next level and stop; nothing could be shallower, so this is optimal from this state and we never expand such a state further. Consequently, every expanded state has $2v + u < s - p$, which bounds the number of reachable states by $O(s^3)$ and transitions by $O(s^4)$.
- **Keep every unused slot internal.** Making $m_{\ell+1} = c - n$ (rather than fewer) can only enlarge later capacity: a state that dominates another componentwise allows a superset of futures, so it is never worse. Childless internals can always be discarded in the last step.

The transition graph is acyclic – $n \geq 1$ increases p , and the $n = 0$ step maps $(u, v) \mapsto (v, 2v + u)$, which strictly increases the capacity – so a plain forward-direction DP in order of increasing p (and, within equal p ,

increasing capacity) suffices. The answer is the smallest cost over all finishing transitions. This is $O(s^4)$ per test with a tiny constant, comfortably covering the $h = 2$ test cases in all remaining subproblems.

To reconstruct a code, store each state's predecessor and chosen n , walk back to recover the per-level leaf counts $n_1, \dots, n_L$, then build the codewords top-down: maintain the list of slot strings arriving at the current level (and, separately, those arriving one level later via *-edges); at each level the first n_ℓ slots become leaves and receive the next symbols in decreasing weight order, while each remaining slot x stays internal and emits $x0, x1$ to the next level and x^* two levels down. Slots on the same level are interchangeable, so any assignment of the required counts works – we can do it simply left to right.